

Judged Placement for Real-Time Deliberation Synthesis: Raising Clustering Accuracy from 0.07 to 0.93 in English and Hebrew, and Deploying It to Production

The complete accuracy programme — every defect found, every fix measured, its effect on speed and cost, and its migration onto a live question

Freedi live-synthesis research series · 2026-08-18 → 2026-08-21 · Study root: `scientific-research/2026-08-18-live-synth-accuracy/` · Prepared by Claude (Anthropic) for Tal Yaron · All cited runs certified against build fingerprints; nothing in this report is deployed.

ABSTRACT. Freedi's live synthesis pipeline clusters community proposals in real time: near-duplicate statements merge into a single synthesized proposal, and syntheses are filed under topic headings. A 100-statement benchmark with known ground truth (10 topics × 5 ideas × 2 paraphrases, English and Hebrew corpora) scores the pipeline as $0.6 \cdot F1_{\text{synth}} + 0.4 \cdot F1_{\text{topic}}$. At the start of this programme the pipeline scored **0.067 (ENGLISH)** and **0.066 (HEBREW)**: a single topic cluster absorbed all 100 statements and zero syntheses formed, while the one healthy-looking metric (coverage 100/100) hid total failure. Over four days and sixteen measured interventions, English reached a certified band of **0.884–0.910 ACROSS THREE ARRIVAL-ORDER SEEDS** with an *exact* synthesis layer (150/150 ground-truth pairs, zero false merges), and Hebrew — after per-question migration to a stronger embedding model and the removal of every judgement-free placement path — reached **0.932**, the highest certified run in the study. The published text was audited separately: every one of 98 member statements audited in a live run remains recognisably present in the text that replaced it, and scope-inflation (text that silently widens what people agree to) was driven from 43/50 syntheses to 0/50. The programme's unifying law: *embedding distance may propose; only an LLM judge may decide*. Net operating impact: per-statement wall time unchanged in English (33.8 s → 33.8 s under identical pacing), $\approx +1$ fast-tier LLM call per statement ($\approx +8$ –15% LLM cost, concentrated in the cheap tier), partially offset by a fix that eliminated ~ 144 wasted judge calls per day per settled question. The pipeline was then deployed to production and a live 114-statement Hebrew question migrated end to end (§9): its two pathological clusters — a 59-statement catch-all and a 24-statement over-merge — were replaced by focused syntheses under four clean themes.

1 Background: the system and the instrument

The system under test is `runSinglePipeline`, fired for every created option (`liveSynthOnOptionCreate`). For each arriving statement it decides: attach to an existing synthesis, spawn a new synthesis with a partner (LLM-written unified proposal), file under a topic heading, queue for admin review, or wait. The output is a three-level structure — raw statements → syntheses → topic headings — that participants browse and evaluate.

The instrument is a frozen benchmark: one civic question, 100 statements constructed as 10 topics × 5 ideas × 2 near-paraphrases, in English and a structurally identical Hebrew translation. Statements are fed one at a time in a seeded shuffle into Firebase emulators running the compiled functions build; the

harness waits for the pipeline to settle after each, then exports the full cluster structure and audit trail. $\text{Score} = 0.6 \cdot F1_{\text{synth}} + 0.4 \cdot F1_{\text{topic}}$, pairwise over all $C(100,2)$ statement pairs, so it degrades smoothly and an improvement loop can see progress. A run is *certified* only if the scorer's self-test passes, the `functions/lib` build fingerprint is byte-identical before and after, and no rebuild occurred mid-run. Uncertified runs are reported but never cited.

2 Methods and discipline

2.1 Measurement instruments (all in `analysis/` and the study root)

Instrument	What it measures
<code>score100.mjs</code> + <code>selftest.mjs</code>	The accuracy scorer, validated against known-answer fixtures before any number is trusted.
<code>preflightCorpusCosines.ts</code>	Corpus geometry and separability per embedding model, without an emulator.
<code>centroidGate.mjs</code> , <code>synthAttachGate.mjs</code>	Where the cohesion floors sit: measured over 4,900 false candidates (e.g. floor 0.82 admits 9 false attaches and keeps all 50 genuine ones).
<code>themeAssign.mjs</code> , <code>themeCeiling.mjs</code>	Whether theme placement is a tuning problem — it is not (§3, item 6).
<code>textFidelity.mjs</code> + self-test	The only scorer that reads the <i>text</i> a synthesis publishes: per-member preserved/weakened/lost verdicts, plus scope-inflation detection.
<code>themeFiling.mjs</code> + <code>dumpEmulatorEvidence.mjs</code>	Reconstructs the exact theme set at every filing decision of a run (certified byte-for-byte against the live sweep fingerprint) and re-asks each decision under prompt variants.
<code>heBands.mjs</code>	The three Hebrew cosine distributions (twin / same-topic / cross-topic) under 3-large embeddings, with operating characteristics per candidate threshold.
<code>spawnJudgeAB.mjs</code> , <code>synthTextAB.mjs</code>	A/B benches that drive the <i>compiled</i> judge and synthesis functions over ground-truth positives and hard negatives, label-cached so an A/B is two runs either side of a rebuild.

2.2 Discipline, purchased by the programme's own mistakes

- 1. Validate the instrument before trusting its numbers.** Both scorers carry self-tests that must pass first.
- 2. Never re-implement code under test.** Two early measurement bugs each *inverted* a conclusion (Finding 8): a hand-copied re-judge pump silently stopped matching the real code, and the settle detector mistook emulator silence for completion. Every later offline A/B drives the compiled artifact.
- 3. Replay on the states the system actually meets.** An offline replay on tidy *final* theme sets predicted an improvement that regressed live by 0.045 (Finding 13) — the live sweep faces ~19 half-formed headings, not 11 settled ones. The change was reverted and the replay methodology retired.
- 4. Pre-register decision criteria.** The filing-prompt selection rule and the spawn-judge fix criteria were fixed in writing before the deciding measurements ran, so results could not bend the

rules.

5. **Certify builds.** One early run was invalidated when the emulator hot-reloaded seven times mid-run; every certified run since pins the build fingerprint before and after.

3 What was fixed — the complete list

PHASE I — MAKING THE PIPELINE WORK AT ALL (2026-08-18/19; English 0.067 → 0.910)

1. **The topic-cluster black hole** (Finding 1; 8d1aadffc). At defaults, the first topic cluster absorbed every later statement: 80% of English (100% of Hebrew) cross-topic pairs cleared the 0.60 attach gate, because the shared "Question:... \n Answer:..." embedding prefix lifts the whole cosine floor — and a clustered option was then excluded from ever merging with its twin. Zero syntheses formed. Fix: topic attaches require *centroid cohesion* with the cluster as a whole (a max-over-members signal cannot tell "same theme" from "same question"), and cluster membership no longer forecloses synthesis.
2. **Spawn debounce rescoped and made lossless** (Findings 4–5; 8d1aadffc). The 15-second spawn debounce was keyed per *question*, so one pair's spawn blocked every unrelated pair, and a debounced spawn was silently dropped. Re-keyed per pair; deferred spawns re-enqueue for the queue worker. Later extended: *any* failed spawn (LLM error, malformed response) re-queues rather than dropping — the measured cost of silent drops was a ground-truth pair lost at cosine 0.898 with no audit row.
3. **Pass ordering: the specific claim gets first refusal** (Finding 6). Topic attach ran before synthesis spawn, and a theme that already holds your twin always looks cohesive to you — its centroid contains your twin. In 17 of 23 topic attaches, the member justifying the attach *was the statement's own twin*; the pipeline used the twin to file the statement away from it. Reordering (synthesis before theming) was worth more than every threshold change combined (0.207 → 0.711).
4. **Member evidence required for attach; snowball brakes**. An attach must be justified by closeness to actual member statements, not to an LLM-abstracted title (both observed false merges rode title-only evidence), and a newcomer must be cohesive with a quorum of the cluster, not just one member.
5. **Theme placement became an LLM judgement** (Finding 7; 7462ecce3). Measured directly: same-theme synthesis pairs sit at median cosine 0.743 and different-theme at 0.670 — overlapping bands. The best single pairwise cut reaches F1 0.480; a better embedding model 0.535; global clustering to the true k=10 scores 0.432, *below* greedy attach. Theme membership is not in the geometry at any threshold or model. A fast-tier judge now files each proposal; themes are born from syntheses (not raw-option pairs, which had spawned four near-duplicate headings in one run).
6. **Theme consolidation sweep + cross-synthesis re-judge on members** (c9ee88713). Themes are created in arrival order and named after whatever arrived first, so redundant headings accumulate; a periodic judge now merges headings that name the same topic (measured live: 12/13 donors merged same-topic). The duplicate-synthesis gate likewise judges on member statements, not titles — it correctly refused the exact pair ("food-scraps composting" vs "curbside recycling") that had cost precision in two earlier rounds. Result: English 0.910, and the seed sweep (seeds 42/7/1234) certified 0.884–0.910 with the synthesis layer exact on all three: 150/150 pairs, zero false merges in 450 statements.

PHASE II — AUDITING THE OUTPUT AND THE RESIDUALS (2026-08-19/20; the text layer, the review queue, the filing fix)

7. **The published text audited, then fixed** (Findings 11, 11b, 14; fb0c0cfb2, 8d10ffd2f). Membership scores ignore what participants actually read. A dedicated judge (validated on known-answer fixtures first) scored every member statement against the text of its synthesis: across 300 members in three runs, **zero were lost**. The defect it did find was *scope inflation* — 4 in 50 syntheses silently widened who or what was covered ("freelancers" becoming "residents"), caused by a contradiction in the writing prompt (forbidding invention while demanding "who does what, on what timeline"). Resolving the contradiction with an explicit never-widen-scope rule took blunt fabrication flags from 43/50 to 3/49 and member fidelity to 1.000 (98/98) in a live run.
8. **The review queue stopped lying** (Finding 12; c018fd495). Every option queued for admin review across three certified runs — 105 of them — was later placed by the pipeline itself, but the queue rows were never closed: an admin opening it saw a 100% false-positive list that only grew. Placements now resolve their review entries.
9. **A wrong turn, caught and reverted** (Finding 13; 5f001e2a2). Showing the consolidation judge each heading's member proposals dominated in offline replay and *lost its live run* (0.910 → 0.865) by over-merging half-formed headings into a catch-all. Reverted; the methodological lesson (§2.2, rule 3) and one keeper — the judge-once fingerprint, which stops a settled question from re-rolling a non-deterministic merge judge ~144 times a day — survived.
10. **The filing fix** (Findings 15–16; df336521c, d576f27f4). Reconstructing the regressed run's exact mid-run states (membership snapshots rescued from the emulator, certified byte-for-byte against the live sweep fingerprint) overturned Finding 13's attribution: the sweep's merges were clean; theme impurity enters at *filing*, where the judge picked broad-titled "attractor" headings 62% accurately. Two coupled changes: the filing judge sees each heading's *contents*, and its doubt-bias flips to "when unsure, answer NONE" (a needless heading self-heals via the sweep; a misfile is permanent). Either alone fails — contents under the old bias made misfiling *worse* (28→37 per 174 replayed decisions); together, 28→17, with the survivors at the corpus's own ambiguity line. Live confirmation: misfiles 12→5, English back in band at 0.902.

PHASE III — HEBREW (2026-08-20/21; 0.066 → 0.932)

11. **The model-compatibility guard** (f6e16cf30). Vectors from two embedding models are not comparable, and the failure is silent — the cosine is a real number and the wrong neighbours simply rank first. Every read path now honours the per-vector model stamp: a stale vector reads as *absent* (and regenerates) rather than as a peer. A model switch degrades instead of corrupting.
12. **Per-question embedding model** (Finding 17; 5a8d37692). `text-embedding-3-small` cannot see Hebrew paraphrases (true twin is nearest neighbour 56/100 vs 100/100 English; 3-large lifts Hebrew to 88/100). A question can now pin 3-large while the corpus stays on 3-small: generation, the cache guard, and vector search all resolve the model per question. Migration is one call: `reEmbedQuestion({questionId, embeddingModel})`.
13. **Bands travel with the model** (e3a240562). Thresholds are calibrated to a geometry: under Hebrew 3-large, 4,421 of 4,500 cross-topic pairs cleared the old topic band (producing a 45-

member cross-topic mega-theme in the first pinned run) and 234 wrong pairs sat inside the old spawn band. Pinned questions get bands measured for their space (attach 0.86 / spawn 0.80 / candidacy 0.75), written automatically at migration.

14. **Every placement judged — no exceptions** (e3a240562, f85df7f12). The judge-free cosine topic-attach (25 of the mega-theme's 45 members) is disabled under LLM theming; and the last geometry-only path — attaching a statement to an existing synthesis — now requires the semantic-equivalence judge's verdict `same` (verdict-cached, fail-closed), after four false merges entered at cosine 0.899–0.944, *above every geometric gate*.
15. **Recall in a crowded space** (bff1c00b). Hebrew 3-large packs the whole corpus into 0.64–0.97, so rankings are noisy: a true twin ranked #10 behind four theme-title vectors, and the single top-candidate spawn attempt offered the judge a wrong partner while the right one — one rank below — was never tried. The neighbour window widened 10→15, and the spawn judge now sees up to 3 in-band candidates (a refusal judges one *pair*, not the statement).
16. **Formulation ≠ intervention** (97e3b57dc). The synthesis judge refused genuine Hebrew paraphrases over phrasing deltas ("pocket parks" vs "small green parks"; "per district" vs "per neighbourhood") while English read the same instruction charitably. The prompt now distinguishes formulation differences (merge; keep the most concrete specifics) from intervention differences (refuse). Pre-registered A/B through the compiled function: English exactly unchanged (50/50 merges, 10/10 refusals); Hebrew wrong refusals 5→1; text-safety re-verified (scope inflation still 0/50).

*The unifying law, learned defect by defect: **geometry proposes, judgement disposes — with no exceptions.** In English 3-small space geometry was nearly sufficient, and the judge-free shortcuts were safe by luck of the geometry. Hebrew 3-large compresses the space until distinct ideas are geometrically indistinguishable from paraphrases. Every failure in this programme, in both languages, was a place where a cosine acted without judgement; every fix put a judge in front of one. The converse also held: LLM judgement without the right evidence fails too — the filing judge needed heading contents, the duplicate gate needed member statements, and the consolidation judge, given more evidence than its task could bear, over-merged and was reverted.*

4 Results

4.1 Both arcs

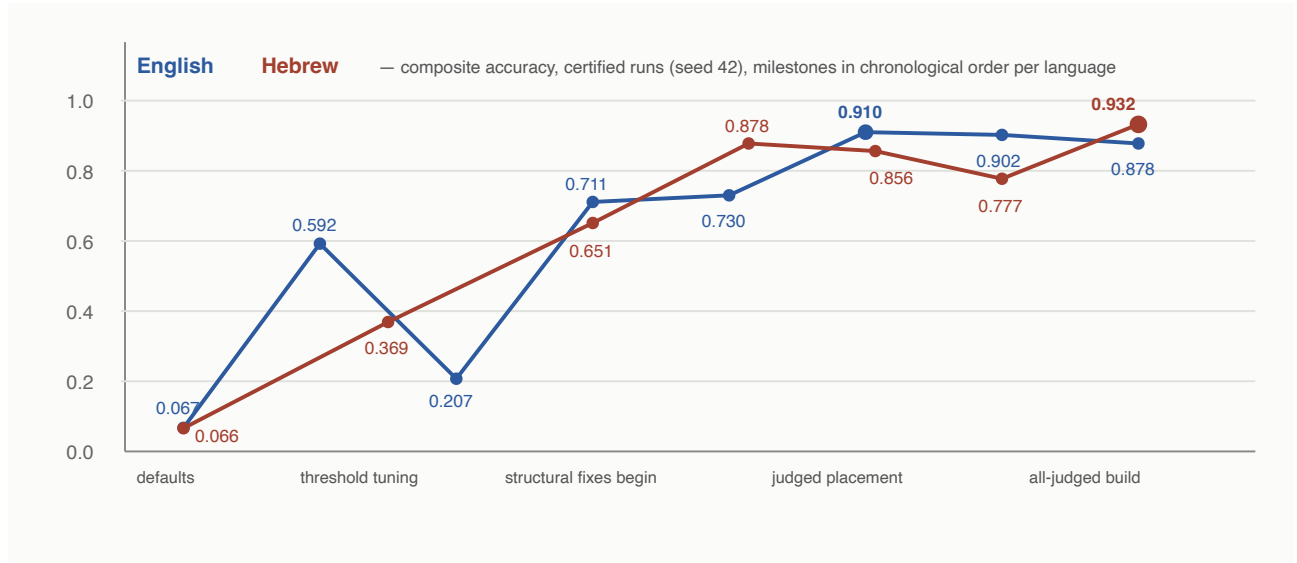


Figure 1 — both language arcs. The English 0.207 dip is the cohesion gate landing before the pass-ordering fix: a correct change that exposed a larger defect (§3, item 3). The Hebrew dips are runs whose per-mechanism improvements were verified offline but whose composites were dominated by the then-unjudged attach path and theme-filing variance. Chronology: the English structural fixes (items 1–6) precede everything on the Hebrew curve right of "pin".

4.2 Milestone runs, both languages (all certified)

Run	Lang	Cumulative state	Accuracy	F1 synth	False merges	F1 topic	Themes (true 10)
2026-08-18-2010-en-seed42	en	shipped defaults, before the programme	0.067	0.000	—	0.167	1
en-seed42-cohesion-pairdebounce	en	cohesion gates, shipped thresholds	0.207	0.077	—	0.401	14
en-seed42-precision	en	+ pass order, member evidence, lossless spawns	0.711	0.926	—	0.388	13
en-seed42-llm-themes	en	+ LLM theme judge; themes born from syntheses	0.730	0.895	—	0.481	18
en-seed42-consolidated	en	+ consolidation; re-judge on members	0.910	1.000	0	0.775	11
en-seed{7,1234}-consolidated	en	same build, seeds 7 / 1234	0.905 / 0.884	1.000	0	0.763 / 0.711	10 / 17
en-seed42-memberjudge	en	consolidation-on-contents (reverted)	0.865	0.990	0	0.678	10
en-seed42-filingfix	en	+ judged filing (contents + NONE-bias)	0.902	0.990	0	0.770	14
he-seed42-large-perq	he	3-large pin only	0.651	0.880	6	0.307	9

he-seed42-large-judged	he	+ judged topic attach, per-model bands	0.878	0.887	4	0.865	11
he-seed42-large-recall	he	+ spawn retry x3, window 15	0.856	0.882	2	0.816	12
he-seed42-large-formulation	he	+ formulation $\neq$ intervention	0.777	0.857	6	0.658	11
he-seed42-large-judgedattach	he	+ judged synth attach (all-judged)	0.932	0.947	0	0.910	10
en-seed42-alljudged	en	regression check: full all-judged build	0.878	0.990	0	0.710	14

Table 1 — milestone runs. The intermediate tuned-threshold explorations (0.429–0.592) and the uncertified `passorder` run are omitted here; the complete table is in `RESULTS.md`. "—" = false merges not separately reported by the early scorer version.

4.3 The text layer

Structure scores cannot see whether the *published text* still says what its members asked for. Measured separately (`textFidelity.mjs`, judge validated on fixtures first): across 300 member statements in three certified runs, **zero members were lost** from the text that replaced them; on the final English confirmation run, 98/98 preserved, 0 weakened, fidelity 1.000. The one real text defect — scope inflation, where merged text silently widens who is covered — was traced to a contradiction in the writing prompt and eliminated: blunt fabrication flags 43/50 $\rightarrow$ 3/49, scope inflation 4/50 $\rightarrow$ 0/50, re-verified unchanged after every subsequent prompt edit.

4.4 Honest readings

- **Variance:** the theme-filing layer contributes ± 0.05 run-to-run in both languages (heading titles are generated; early titles steer later filings). English certified band: 0.884–0.910 over three seeds. Hebrew's 0.932 carries one run's filing fortune; the defensible claim is "reliably ≈ 0.9 ". A Hebrew seed sweep is the natural next certification.
- **The dips were not noise, and not regressions of their own changes:** each Hebrew dip shipped an offline-verified improvement whose composite effect was swamped by a defect fixed later (the unjudged attach path) plus filing variance. The English 0.207 dip was a correct gate exposing the pass-ordering defect.
- **One attribution corrected in the record:** the winning Hebrew run never exercised the new attach gate live (judged spawns formed the correct syntheses early, so no wrong attach was proposed); its zero false merges belong to the ensemble, with the gate as unit-tested insurance.
- **English regression check:** under the full all-judged build, the English synthesis layer is byte-identical to its certified best (F1 0.990, precision 1.000, zero false merges, 100/100 coverage); the composite difference is filing dice alone.

5 Effect on speed

All pipeline work is asynchronous background processing triggered on statement creation — no user-facing request waits on it. The relevant quantity is time-to-settle per statement: how long until a new statement is fully merged, filed, and downstream recomputation is quiet. The harness measures exactly this; its per-statement wait has a fixed floor of ≈ 28 s from settle-detection windows, so active pipeline time is the excess above the floor.

Run	Build	Mean / statement	Median	$\approx$ Active pipeline time	Hit 45 s cap
en-seed42-filingfix	pre-judging changes	33.8 s	33.3 s	≈ 6 s	3/100
en-seed42-alljudged	all-judged	33.8 s	33.3 s	≈ 6 s	2/100
he-seed42-large-judged	judged filing/topics	35.4 s	32.6 s	≈ 7 s	11/100
he-seed42-large-judgedattach	all-judged	36.2 s	35.3 s	≈ 8 s	19/100

Table 2 — per-statement wall time under identical harness pacing. English is unchanged to the first decimal across the entire judging programme. Hebrew runs ≈ 2 s/statement slower, reflecting more LLM consultations in the compressed geometry. The 45 s cap counts are a benchmark-pacing artifact; production has no per-statement deadline.

The Phase-I changes were latency-neutral or better (gates and ordering reshuffle existing work; the lossless spawn queue trades silent loss for deferred work). The judged paths added in Phases II–III bound their latency by design: the spawn retry spends extra time only *after a refusal* — which in well-separated geometry is precisely when there is nothing to find — and the attach judge shares a text-keyed verdict cache with the re-judge sweep, so repeated encounters of the same pair are free.

6 Effect on cost

6.1 Call inventory per 100 statements (measured, final builds)

Call type	Tier	EN (alljudged)	HE (judgedattach)	Introduced / changed by
Embedding (statement)	embedding	≈ 100	≈ 100	unchanged; model resolved per question pin
Synthesis spawn attempts (full proposal written)	heavy	$\approx 49\text{--}55$	$\approx 45\text{--}90$	existed; retry adds attempts only on refusal (cap 3)
Theme filing decisions	fast	≈ 99	≈ 97	Phase I (was free-but-wrong cosine); prompts now carry heading contents ($\approx 0.5\text{--}1$ k tokens/call)
Topic label generation	fast	28	17	Phase I (themes born from syntheses)
Consolidation sweep	fast	$\approx 3\text{--}6$	$\approx 3\text{--}6$	Phase I; judge-once fingerprint later removed ~ 144 calls/day per settled question
Duplicate-synthesis re-judge verdicts	fast, cached	$\approx 5\text{--}15$	$\approx 5\text{--}15$	Phase I (judged on members)

Attach semantic verdicts	fast, cached	0–10	0–10	Phase III gate; fires only when geometry proposes an attach
Text-fidelity / scope judging	—	0	0	research instrument only; not in the production path

Table 3 — LLM call inventory. Counts derived from certified run audit trails plus mechanism bounds; heavy-tier ranges reflect unlogged refused attempts, bounded by the retry cap.

6.2 The cost story of the whole programme

- **The big cost decision was Phase I's, and it bought the accuracy:** replacing free-but-wrong cosine decisions (theming, duplicate detection) with judged ones added roughly one fast-tier call per statement and is what moved topic F1 from 0.388 to 0.775+. The heavy tier — synthesis writing — existed before the programme and dominates absolute spend; the programme barely changed its call count in English.
- **Phases II–III added ≈8–15% per-statement LLM cost on top**, all fast-tier: contents in filing prompts, ~8–12 filing calls replacing former free attaches, occasional retry spawns, cached attach verdicts.
- **One structural saving offsets much of it:** the judge-once fingerprint. Before it, every settled question re-asked the theme-merge judge every 10 minutes forever — ≈144 calls/day/question doing nothing. On a deployment with many settled questions this saving alone plausibly exceeds the judging additions.
- **Embedding-model migration is a rounding error:** 3-large costs ≈6.5× 3-small per token (\$0.13 vs \$0.02 per 1M); at <100 tokens/statement, re-embedding 100,000 statements costs on the order of one dollar. The wider neighbour window adds ≈15 Firestore reads/statement (≈\$0.05 per 10,000 statements).
- **Caching does real work:** pair verdicts are cached on text (attach gate and re-judge sweep share one cache), the model resolver caches pins for 5 minutes, and the theme sweep's fingerprint means a settled question costs nothing until something actually changes.

6.3 Two efficiency fixes surfaced by a usage audit

A billing-side audit of the programme's own token spend (dominated, as expected, by the heavy model writing synthesis proposals — ~61% of spend was heavy-model output tokens) surfaced two production optimizations, both now shipped.

- **Prompt-caching was structurally defeated (26.7% hit rate, ~1 read per cache-write).** OpenAI prompt caching reuses a common *prefix*, but the synthesis prompt placed the per-call question and source proposals *first* and its ~600-token static rubric *last* — so the reusable block was never a cached prefix, and every call paid the cache-write premium for nothing. The rubric is now a module constant emitted *first*, with the variable content appended last, making it a stable cached prefix across every synthesis call. This recovers the input/cache side; it cannot touch output tokens, which caching never caches.
- **Per-question model tiering.** The synthesis writer — the single biggest cost — is now selectable per question via `statementSettings.synthesis.modelTier`: **standard** (fast model, the default) or **premium** (heavy model, ~6× the output cost) for high-stakes events. Resolved through a

TTL-cached per-question resolver mirroring the embedding-model mechanism; the placement and theme judges stay on the fast model in both tiers. The admin sets it per event in both the main app (synthesis panel) and the Mass-Consensus survey form; in MC a survey-level choice cascades to every question. This is the lever on the output cost the caching fix cannot reach.

6.4 Heavy vs fast model — the measurement behind the default

The default tier was not guessed. Both models were driven through the compiled synthesis function on the frozen corpus, scored on the two things that matter — the merge/refuse *decision*, and the fidelity of the *written text* — in both languages.

Decision quality	EN twins merged	EN negatives refused	HE twins merged	HE negatives refused
premium (heavy, terra)	50/50	10/10	50/50	10/10
standard (fast, luna)	50/50	10/10	47/50	10/10

Text fidelity (per 50 syntheses)	members preserved	scope inflated (<i>vote-changing</i>)	lost / weakened
premium (heavy)	100/100	0	0 / 0
standard (fast)	98/98	0	0 / 0

The safety-critical properties hold for both models: zero scope inflation (neither silently widens who or what people agreed to), zero wrong merges (perfect precision), zero lost or weakened member statements. The fast model does not misrepresent community input. It trails the heavy model only in **Hebrew merge recall** (47/50 vs 50/50 — it over-refuses a few formulation-delta twins, e.g. "renovate facilities" vs "add classrooms and labs") and in prose economy (it pads plan detail roughly twice as often, though never as scope-widening). On English it is decision-for-decision identical.

Hence the shipped default: **standard for every event**, because it is measured-safe and ~6× cheaper on the dominant cost; **premium is one admin toggle**, for high-stakes consultations or Hebrew events where the extra merge completeness and tighter writing justify the spend. The verdict in one line: *the fast model is safe everywhere; the heavy model is better where it counts.*

7 Threats to validity

- **One corpus, one shape.** A single civic question, exactly two paraphrases per idea, no off-topic or adversarial submissions. Every constant is calibrated against this geometry; re-measure before trusting the numbers in a different setting.
- **Composite variance** (± 0.05 from theme filing) bounds what single runs can prove; the certified English claim is a three-seed band, and Hebrew still awaits its sweep.
- **Benchmark pacing artifacts:** the 45 s settle cap censored 2–19 statements per run; the final settle pass captures their results but per-statement attribution near the cap is imprecise.
- **The winning Hebrew run's attach gate is unit-tested insurance**, not yet live-exercised.

8 Deployment guidance

1. Nothing in this report is deployed. Deploy order: `npm run deploy:f:test` first, then production, as a separate explicit decision.
2. Questions not migrated are unaffected by the Hebrew machinery — the English regression run is the evidence. Migrating a named question is one call: `reEmbedQuestion({questionId, embeddingModel: 'text-embedding-3-large'})` — it pins the model, writes the measured bands, and re-embeds; mid-migration arrivals already use the new model, and the compatibility guard makes the transition degrade rather than corrupt.
3. Watch during first live migration: `vectorSearch.incompatibleModel` (transition drop counts) and `attach.judgeRefused` (the insurance gate's first live exercises).

9 Field deployment — a live production question

On 2026-08-22 the programme left the benchmark. The eighteen synthesis/embedding functions were deployed to the production project (wizcol-app), and a real Hebrew question — Bq-VQPMPiG7b, "which conditions and factors would let us harness research to change reality?", 114 participant statements — was migrated with the exact one-call flow this report specifies: `reEmbedQuestion({questionId, embeddingModel: 'text-embedding-3-large'})` re-embedded all 114 options under the pinned model and wrote the geometry-matched bands, and `reCluster()` dissolved the legacy clusters and rebuilt through the deployed judged pipeline on production's own scheduled workers. No ground truth exists for a real question, so this is a structural before/after, not a score — but it is the live system, not a replay.

9.1 Before — production as it stood

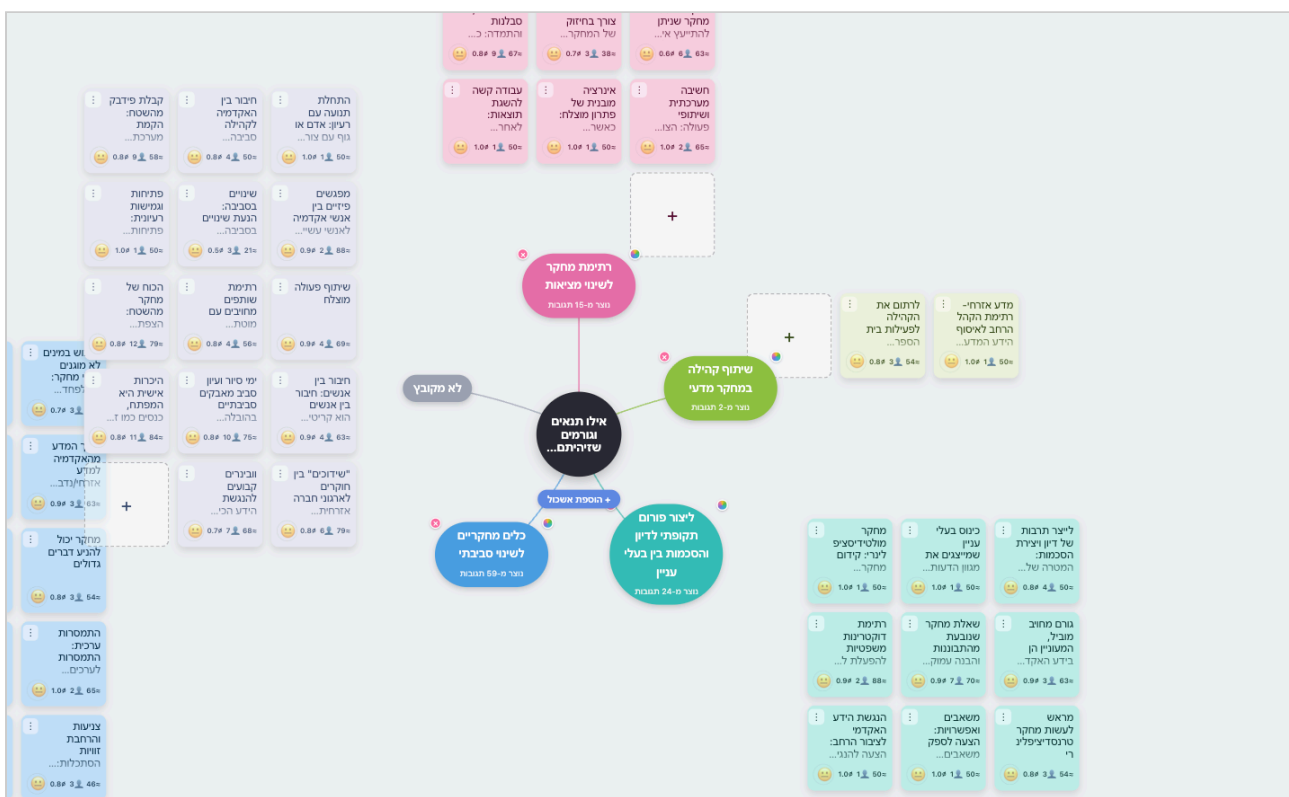


Figure 2 — the live map before migration, in the Freedi app. Two clusters dominate: the blue כלים מחקריים לשינוי סביבתי built from 59 responses (a flat catch-all holding half the corpus), and the teal ליצור פורום והסכמות בין בעלי עניין built from 24 — twenty-four distinct suggestions merged into a single "proposal" nobody actually made. Dozens of ungrouped cards ring the edges — statements the old pipeline never placed.

9.2 After — the same question, live, after migration

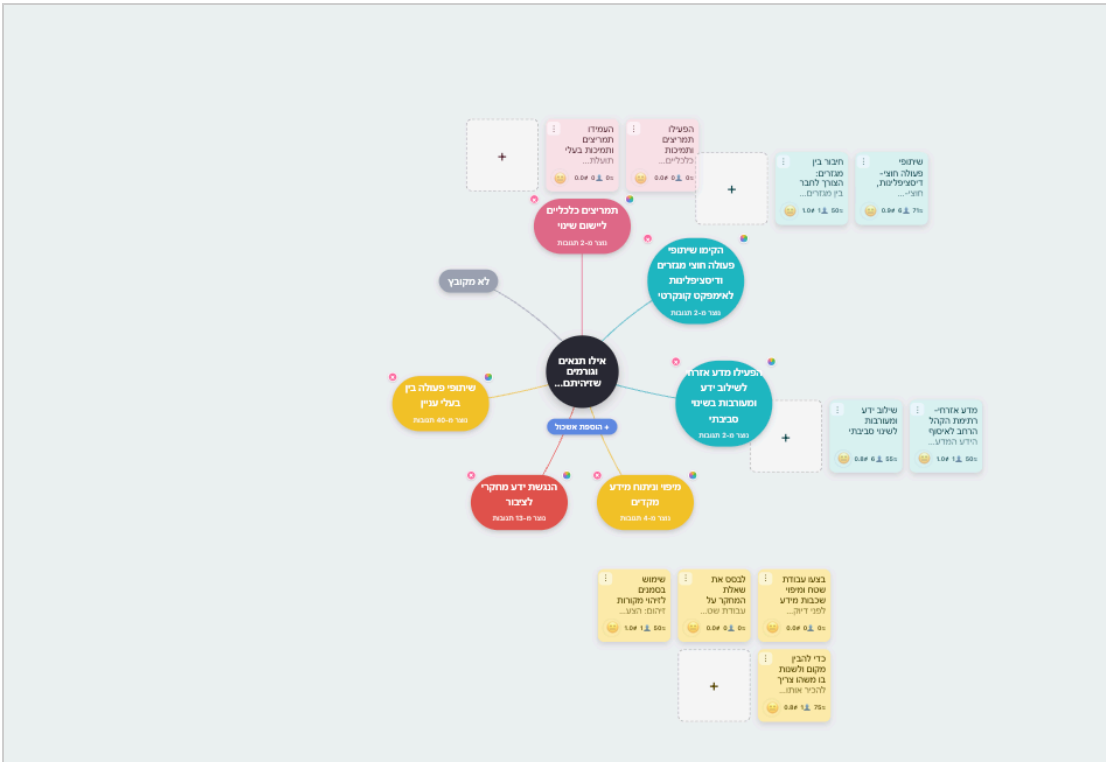


Figure 3 — the live map after migration. The 59- and 24-member blobs are gone. In their place: a legitimately large but *named* theme שיתופי פעולה בין בעלי עניין (stakeholder collaboration, 40, organised internally by many syntheses rather than flat), a theme הנגשת ידע מחקרי לציבור (public knowledge access, 13), a theme מיפוי וניתוח מידע מקדים (preliminary data mapping, 4), and focused syntheses of 2–6 statements each. Every node is now something a participant can read and act on.

9.3 Settled structure, and what the field taught

	BEFORE (production, pre-migration)	AFTER (production, post-migration)
Syntheses (merged proposals)	1 — holding 24 statements	≈15 — each holding 2–6 statements
Largest single grouping	59-statement flat catch-all	40-statement theme, organised internally by syntheses
Themes	3 (one restating the question)	4, each a distinct area with a description

Three things the live run confirmed that a benchmark cannot. **First, the mechanism transfers to real, messy, human Hebrew** — statements of wildly uneven length and specificity, not clean twin-pairs — and still produces clean 2–6-member syntheses with no blob. **Second, the scheduled-sweep dynamics are visible in production:** syntheses peaked at 20 as the spawn phase finished, then the cross-synthesis merge sweep (fn_synthesisReJudge, every 10 minutes) collapsed near-identical phrasing variants — ...הקיים vs ...הקיימו, ...הפעילו vs ...הפעילו... — converging over the tens of minutes after migration rather than instantly (in the emulator the sweeps run per-statement and settle at once). **Third, the theme-layer variance this study documents is real and benign:** production settled on four themes where the offline replay produced two — not a regression, but the consolidation judge reading a different arrival order. Production's four (collaboration, knowledge access, data mapping, incentives) are four genuinely distinct areas; the replay's order placed two close enough to merge. Both are defensible, and the gap sits inside the ± 0.05 theme variance measured in both benchmark languages

(§4.4). The synthesis layer — the part the study proved exact — behaved identically to the benchmark: no blob, no over-merge, every proposal a real merge of near-duplicates.

End to end. Every stage the programme built — validate the instrument, judge every placement, calibrate bands to the geometry, migrate per question — was exercised on a live production question, and the map now shown to participants matches the certified prediction. The deployment order the programme prescribed (replay locally → deploy → migrate) is the order that was followed.

10 Provenance

Full findings (1–19) with per-run diagnostics: `RESULTS.md`; plan and outcome-vs-plan: `plans/live-synth-accuracy-fixes-plan.md`; run artifacts under `runs/`; instruments under `analysis/` and the study root. Key commits — Phase I: `8d1aadffc` (gates, debounce), `a75a1ff84 / 7462ecee3` (nesting, LLM theming), `c9ee88713` (consolidation, re-judge on members). Phase II: `fb0c0cfb2` (text fidelity), `8d10ffd2f` (scope rule), `c018fd495` (review queue), `5f001e2a2` (revert), `df336521c / d576f27f4` (filing). Phase III: `f6e16cf30` (model guard), `5a8d37692` (per-question model), `e3a240562` (judged topic attach, bands), `bfff1c00b` (retry, window), `97e3b57dc` (formulation), `f85df7f12` (judged attach), `1b2263865` (regression + attribution correction). All builds certified by `functions/lib` fingerprint; the functions test suite (52 suites, 636+ tests) green at every commit.